

Build a Firecube plugin for your dataset

Goal: Learn and understand the concepts of Firecube, work towards an example plugin for your selected dataset. Go home with a working plugin and a clear understanding on the next steps to make datacube creation operational with Firecube.

Before you arrive

- Install Firecube using the [installation guide](#). Confirm the environment with `firecube --version`.
- Bring a dataset you want to convert to a datacube, preferably in NetCDF.
- Please note: You will work on your own laptop, in your own compute environment.
- If you use AI coding assistants, you can use the Firecube LLM skills, which are available for the event.

Agenda

Time	Activity	Result
08:30–09:00	Roundtable and introductions	Know the participants, datasets, and Firecube
09:00–10:00	Run the Firecube quickstart	Quickstart plugin installed and verified, sample Zarr cube. Time for a Q&A.
10:00–10:30	Coffee break	
10:30–12:00	Create your own plugin	Progress through the milestones below
12:00–12:30	Demonstrations, feedback, and awards	Short demonstration of your result; awards for the winners and feedback

Run the Firecube quickstart

Run a complete example plugin from installation to a Zarr cube before building your own plugin.

1. Follow the [Quickstart Guide](#).

Done when: the quickstart plugin is listed and described, the ingestion results with a Zarr cube, and cube shows the expected dimensions and variables.

Help: follow [Verify the installation](#). If a command is missing, check the active environment. If the plugin is not detected, reinstall it in the same environment as Firecube or show the error to your instructor.

Milestone 0: create your plugin repository

Generate the plugin scaffold with the Firecube CLI, following [Create a plugin](#).

1. Choose the template that fits your data: `zarr` for gridded arrays, `parquet` for tabular or point data.
2. Run the interactive command and supply your own plugin name, author, and licence at the prompts:

```
firecube plugins create <your_plugin_name>
```

3. Open the generated `firecube-<your_plugin_name>/` directory. Locate `README.md`, `pyproject.toml`, and the ingestor module in `src/`, with its `PRODUCT_NAME` declaration and the reader hook (`read_dataset` or `read_table`, according to the template).

Done when: the scaffold exists and you know where your reader implementation belongs. See [Verify the created project](#).

Milestone 1: install and discover your plugin

Give Firecube a package it can load, following [Install your plugin](#).

1. From the plugin directory, install it into the same environment as Firecube:

```
firecube plugins install --editable .
```

2. Check that Firecube discovers it with `firecube plugins list`.
3. Inspect it and its available options with `firecube plugins describe <your_plugin_name>`.

Done when: Firecube lists and describes your plugin. Ingestion comes after the reader is implemented.

Hint: the package exists, but Firecube cannot find it

Creating a directory does not install the plugin. Check the installation environment and registration metadata using the installation guide. The generated README is the starting reference for the selected scaffold.

Milestone 2: read your data

Set `TIME_DIM` and populate the `read_dataset` function. Ingestion stops with `NotImplementedError` until both are done.

1. Set `TIME_DIM` in the ingestor module to the name of the time dimension in your files, for example `TIME_DIM = "time"`. Firecube appends every batch along it.
2. Implement `read_dataset` so it returns one source file as an `xarray.Dataset`. For NetCDF, uncomment the example in the scaffold; for other formats, use the reader you already trust and load the data before the file closes.
3. If your files do not have a time dimension, create one in `read_dataset`. When the time exists only in the filename, parse it with `parse_pattern` (install it first with `uv add 'firecube[patterns]'`), following [Parse filename fields](#).
4. If your data does not arrive as one plain file per time step, pick the guide for your layout:
 - `.zip` archives: [Discover zipped data](#)
 - two files per time step, such as data plus metadata: [Read paired source files](#)
 - files Firecube does not find, or files to exclude: [Discover source data](#)
5. Keep only the variables you need, with their coordinates, units, fill values, and attributes.

6. Call `read_dataset` directly from Python and print the results. The editable install from Milestone 1 makes your package importable in the same environment:

```
from pathlib import Path

from firecube_<your_plugin_name>.ingestor import read_dataset

for path in [Path("/path/to/sample_01.nc"), Path("/path/to/sample_02.nc")]:
    ds = read_dataset(path)
    print(ds)
    print(ds["<variable>"].isel(<your_time_dim>=0).values)
```

The package name is the `src/` directory name in your scaffold, for example `firecube_my_plugin`. If your `read_dataset` also takes the filename, call it as `read_dataset(path, path.name)`. Save the snippet as a script or run it in a notebook, and rerun it after each change to `read_dataset`.

Read: [GenericZarr: set TIME_DIM and define read_dataset](#), or [GenericParquet](#), according to your track. For a complete worked example, see the [Firecube 101: NetCDF to Zarr](#) notebook.

Done when: `read_dataset` returns a dataset for two different sample files, each with the expected variables, coordinates, and time dimension, and one value you checked against the source file.

Milestone 3: implement build_dataset

`build_dataset` turns one batch of source files into one dataset that Firecube appends to the cube. The scaffold version reads every item with `read_dataset`, concatenates along `TIME_DIM`, and sorts on time. Adapt it to your data.

1. Read the generated `build_dataset`. It works as is when `read_dataset` takes only the file path. If you followed one of the guides in Milestone 2, make the `build_dataset` change that guide shows.
2. Make every batch consistent: the same variables, dimensions, coordinates, and data types, so later batches can append to earlier ones. Drop per-file attributes such as `history` that differ on every file.
3. Make sure time values within a batch are unique and sorted. If several files together form one time step, group them into one item instead of concatenating duplicates (see [Read paired source files](#)).
4. Return `None` for an empty batch, as the scaffold does.

Hint: my time comes from the filename

Pass the filename to `read_dataset` in `build_dataset`:

```
datasets = [read_dataset(ctx.materialize(item), Path(item).name) for item in items]
```

See [Parse filename fields](#).

Read: [GenericZarr: implement build_dataset](#), its [common mistakes](#), and [the append data contract](#).

Done when: for a batch of two or more sample files, `build_dataset` returns one dataset with unique, sorted time values and a consistent schema.

Milestone 4: ingest one example file

1. Run `firecube ingest <your_plugin_name>` command with your own inputs. Use a fresh local target with a full `file:///...` URI:

```
firecube ingest <your_plugin_name> \  
  --input-data /path/to/one-file-input \  
  --target file:///path/to/<your_plugin_name>_out.zarr \  
  --product-name <your_plugin_name> \  
  --storage-type local \  
  --storage-driver fsspec \  
  --output-format zarr \  
  --write-mode direct
```

2. Check that the run finishes and reports the created product.

4.1: open the cube and validate it

1. Open the cube with `xarray`, using the group your plugin wrote (`default` for the scaffold):

```
import xarray as xr  
ds = xr.open_zarr("/path/to/<your_plugin_name>_out.zarr", group="default", consolidated=False)  
print(ds)
```

2. Check the variables, dimensions, coordinates, time value, and units against the source file. Compare at least one data value, including a missing or fill value if your data has one.
3. Validate the stored structure with
`firecube zarr validate -p file:///path/to/<your_plugin_name>_out.zarr -g default`.

Read: [GenericZarr verification](#) and [open and inspect a cube](#).

Done when: the ingestion creates a cube from one file, the cube reopens with the expected variables, dimensions, and time value, one data value matches the source, and `firecube zarr validate` reports no errors.

Milestone 5: ingest more data

Append later files to the cube you created in Milestone 4.

1. Record the current time values and a few existing data values.
2. Put later, compatible files in a new input directory. Their times must come after the times already in the cube; ordinary append cannot insert earlier dates.
3. Run the same ingestion command against the same target, pointing `--input-data` at the new directory and adding `--option resume_existing=true`.
4. Reopen the cube and check that the new time steps are present, the earlier values are unchanged, and the metadata and coordinates are retained.
5. Rerun the same inputs and check that no duplicate time values appear.

Read: [GenericZarr append](#) and [the append data contract](#).

Done when: the cube contains the original and the new time steps in order, without duplicates, and the earlier values are unchanged.

Milestone 6: implement one customization option

Set a default for one storage setting (chunks, sharding, or compression) in your plugin, then change it at run time with `--option` in the ingest command.

1. Choose one storage setting: chunk shape (`zarr_chunk_shape`), sharding (`zarr_sharding` with `zarr_shard_shape`), or compression (`zarr_compression` or `zarr_codecs`).
2. Set its default in your scaffold: in the ingestor module, uncomment the setting in the `ZarrStorageConfig` class and adjust it to your dimensions. Decide the chunk shape before the first real run, because later appends must match the chunks already on disk.
3. Ingest into a fresh target without any `--option`. The cube uses your scaffold default.
4. Ingest the same inputs into another fresh target and overwrite the default from the CLI, for example `--option 'zarr_chunk_shape={"time":2,"lat":200,"lon":200}'` with your own dimension names and sizes. Quote the whole option, or the shell eats the braces.
5. Compare the two cubes: the chunks, shards, or codecs differ, and the data values are the same. Each guide below ends with a short Verify snippet that prints them.

Hint: why a fresh target every time?

A cube keeps the storage settings it was created with. Ingesting into an existing cube with a different chunk shape fails, even with `resume_existing=true` or `force_reingest=true`. A different compression or sharding setting does not fail: the run succeeds, but the cube keeps its old settings.

To try another setting, use a new `--target` path, or delete the old cube directory first. Firecube keeps its ingestion records inside that directory, so deleting it starts clean.

Your new default also applies to later runs into the cube from Milestones 4 and 5, for example in Bonus 4. If its chunk shape differs, those runs fail; pass its original chunk shape with `--option zarr_chunk_shape=...`, or rebuild that cube.

Read: the guide for your setting: [Configure Zarr chunking](#), [Configure Zarr sharding](#), or [Configure Zarr compression](#). All options and defaults are in `ZarrTemplateConfig`.

Done when: one cube is written with your scaffold default and one with a runtime `--option` override, and you can show the different chunks, shards, or codecs and the matching values.

Bonus 1: create an Intake catalog and use it to read your Zarr cube

1. Install the catalog reader, following [Install the catalog reader](#).
2. Generate a catalog for your cube:

```
firecube catalog intake <your_plugin_name> \  
  --product file:///path/to/<your_plugin_name>_out.zarr \  
  --collection-id <your_collection_id> \  
  --output file:///path/to/catalogs/<your_collection_id>.yaml \  
  --no-storage-options
```

3. Open the catalog with `intake.open_catalog`, select its source, and read the cube.
4. Compare the variables, dimensions, and one value with the cube you opened directly in Milestone 4.

Read: [Create an Intake catalog](#) and [Verify the catalog](#).

Done when: the catalog lists a source for your product and reading it returns the same data as opening the cube directly.

Bonus 2: list the chunks using the chunk manager

1. List the tracked records for your cube, including span coverage:

```
firecube chunks list --product-name file:///path/to/<your_plugin_name>_out.zarr --include -span
```

2. List the ingestion runs with `firecube chunks runs list` and match each run to the inputs you ingested in Milestones 4 and 5.
3. Filter the spans to one time window with `--type span --time-range START:END`.

Read: [Inspect chunk records](#) and [Filter by coverage](#).

Done when: you can show which runs and spans wrote each time range in your cube.

Bonus 3: delete one ingestion run from your cube using the chunk manager

1. Work on a copy of your cube or a cube you can rebuild, and make sure no ingestion is writing to it.
2. Firecube deletes one ingestion run at a time, not single dates: a span covers everything one run wrote. Pick the run that appended data in Milestone 5, using `firecube chunks list --type span --include-span`, and note its run id.
3. Preview the deletion with `--dry-run`:

```
firecube chunks delete-span --product-name file:///path/to/<your_plugin_name>_out.zarr --run-id <run_id> --dry-run
```

4. Run the deletion with `--yes-i-really-mean-it`, then reopen the cube. The time steps stay in the cube, but their values are now empty (NaN); the data from Milestone 4 is unchanged.

`firecube chunks delete --range` filters by record timestamps, not by the times in your data; delete by run span instead.

Read: [Delete storage chunks from spans](#) and [Delete by date range](#).

Done when: the values of the deleted run's time steps are NaN and the data outside that run is unchanged.

Bonus 4: reingest the deleted run using the chunk manager

1. Put the source files of the deleted run in an input directory.
2. Run your ingestion command against the same target with `--option force_reingest=true`.

3. Rebuild the snapshot with

```
firecube chunks snapshots rebuild --product-name file:///path/to/<your_plugin_name>_out.zarr.
```

4. Reopen the cube and compare the restored values with the source and with the values you recorded before the deletion.

Read: [Reingest a range](#).

Done when: the deleted time steps have their values back, the values match the source, and no duplicate time values appear.

Getting help

Raise a hand to get help from your instructor or check the documentation below.

Getting started

- [Installation](#)
- [Install the example plugin](#)
- [Prepare source data](#)
- [Run ingestion](#)

Building your plugin

- [Create a plugin](#)
- [Install a plugin](#)
- [GenericZarr ingestor](#)
- [GenericParquet ingestor](#)
- [Add plugin configuration options](#)

Reading your source files

- [Source discovery](#)
- [Customize source discovery](#)
- [Parse filename fields](#)
- [Discover zipped data](#)
- [Read paired source files](#)

Zarr output and configuration

- [GenericZarr append and data contract](#)
- [Configure Zarr chunking](#)
- [Configure Zarr sharding](#)
- [Configure Zarr compression](#)
- [Configuration reference](#)

Showcase notebooks

- [Firecube 101: NetCDF to Zarr](#): create a cube, append to it, and check for duplicates
- [Sentinel-3 SLSTR fire radiative power](#): real satellite data written to Parquet
- [Slot-based parallelism: MTG FCI L1C](#): parallel Zarr writes with DirectZarr

Operations

- [Create an Intake catalog](#)

- [Inspect chunk records](#)
- [Delete and reingest chunks](#)